Programación Lógica y Funcional

Apuntes de Clase: Haskell (Intervalos, Operadores y Evaluación Perezosa)

Alumno: López Calderón Jerry Emiliano
Unidad 2: Lenguaje Haskell

2.3 Intervalos (Ranges)

En Haskell, los intervalos (o *ranges*) son una sintaxis concisa para generar listas de elementos ordenados en una secuencia de manera eficiente.

Sintaxis básica e intervalos simples

```
-- Lista de números del 1 al 10
delUnoAlDiez :: [Int]
delUnoAlDiez = [1 .. 10]
-- Resultado: [1,2,3,4,5,6,7,8,9,10]

-- Intervalo de caracteres
abecedario :: [Char]
abecedario = ['a' .. 'f']
-- Resultado: "abcdef"
```

Definición de pasos e intervalos infinitos

Si indicas los primeros dos elementos, Haskell calcula automáticamente el patrón de salto:

```
[2, 4 .. 10]        -- Produce pares: [2, 4, 6, 8, 10]
[10, 9 .. 1]        -- Genera lista descendente: [10, 9, 8, 7, 6, 5, 4, 3, 2,
1]

-- Intervalos infinitos: Al omitir el límite superior se crea una secuencia sin
fin
take 5 [1 ..]       -- [1, 2, 3, 4, 5]
take 5 [5, 10 ..]   -- [5, 10, 15, 20, 25]
```

2.4 Operadores y Cambio de Sintaxis

Los operadores en Haskell son funciones infijas que se escriben entre sus dos operandos.

- **Aritméticos:** +, -, *, /
- **Comparación y Lógicos:** ==, /= (desigualdad), &&, | |
- **Listas:** : (agrega un elemento al inicio) y ++ (concatena dos listas)
- **Especiales:** . (composición) y $ (aplicación con baja precedencia para evitar paréntesis)

Cambio de Sintaxis

- Una función normal binaria se convierte en operador infijo usando comillas invertidas (backticks): `10 `div` 2`
- Un operador se convierte en función prefija envolviéndolo en paréntesis: `(+) 5 10`

2.7 Evaluación Perezosa (Lazy Evaluation)

Es la característica central de Haskell donde ninguna expresión se evalúa hasta que su resultado sea indispensable para continuar la ejecución del programa.

- **Thunks:** Las expresiones no calculadas se guardan como "promesas" de cómputo llamadas *thunks*, las cuales solo se resuelven cuando se requiere su valor.
- **Manejo de listas infinitas:** Permite definir listas sin fin y tomar solo los elementos necesarios sin saturar ni agotar la memoria.

```
main :: IO ()
main = do
  putStrLn "-- Evaluación Perezosa --"
  print (take 5 [1 ..]) -- Evalúa únicamente los primeros 5 elementos de la lista i
nfinita
```